

CSCE 713 SOFTWARE SECURITY · MODULE 04 · LECTURE NOTES

Injection and Parsing

One cause, many names, one structural fix.

MODULE OUTCOMES

- Explain injection as a parsing problem.
- Explain why parameterisation works and escaping is fragile.
- Explain path traversal and command injection.
- Explain parse-don't-validate.
- Fix an injection structurally rather than locally.

1 The cause

INJECTION HAPPENS WHEN DATA IS CONCATENATED INTO A STRING THAT IS LATER PARSED AS CODE

The interpreter downstream receives a single string and cannot possibly tell which parts you intended as data and which as structure — because that information was destroyed by the concatenation itself, before the string ever reached the parser.

So every variant is the same defect at a different parser: SQL injection at the database's parser, command injection at the shell's, cross-site scripting at the browser's HTML and JavaScript parsers, template injection at the template engine's, and log injection at whatever reads the log. One cause, five names.

And the reason escaping is fragile is that correct escaping depends entirely on the destination context — which the point of concatenation does not reliably know and which changes when the code is refactored (§2).

Which is why the structural fix is to never build the string at all — use an interface that carries data and structure in separate arguments, so that the interpreter receives them already distinguished and has nothing to guess.

```

SQL      parameterised queries / prepared statements.
        Always. The query structure is fixed before
        any data is bound to it.

SHELL    pass an argument ARRAY, not a string --
        execve-style. Better still: do not invoke a
        shell at all (section 3).

HTML     the template engine's contextual
        autoescaping, or a DOM API that takes text
        rather than markup.

JSON/XML a serialiser. Never string building.

FILESYSTEM resolve the path, then verify the result is
        inside the intended directory. Do NOT filter
        for ".." (section 3).

LDAP/XPath the library's own parameter binding.

THE PRINCIPLE: the interface keeps data and structure in
        separate arguments, so that no parser ever has to
        guess which is which.

```

Every row is the same idea — separate arguments rather than one concatenated string, so that nothing has to be escaped at all. Framing all six as a single principle is the teaching value of this module, because students who learn them as six unrelated remedies apply each one only where they have seen it and miss the seventh parser.

2 Why escaping fails

- **The context changes and the escaping does not.** A value correctly escaped for HTML text is unsafe when placed in an attribute, a URL, a script block, or a CSS value — five contexts with five different correct escapings, and a template change moves a value between them silently.
- **Escaping at the input boundary cannot know the destination**, so it escapes for a guess — which is CSCE 701 Module 06's **escape-at-the-point-of-use argument** and is why boundary sanitisation is the wrong architecture rather than merely an incomplete one.
- **Double-escaping and double-decoding** — a value escaped twice and decoded once, or the reverse, so that **the filter and the parser see different bytes**. This is the mechanism behind a long sequence of filter bypasses.
- **Encoding differences: a filter reading strict UTF-8 while the parser accepts overlong encodings, alternate representations, or a different character set** for the same character — so the dangerous character is present to one and absent to the other.
- **And a blacklist, which is always incomplete** — the attacker needs only the one case you missed, and the space of cases is defined by someone else's parser. **Escaping is not wrong; it is *positional*** — correct only relative to a destination — **which is exactly why it belongs at the point of use and inside the template engine**, where the destination is known.

3 Paths and commands

FOR PATHS, RESOLVE FIRST AND THEN CHECK CONTAINMENT

Filtering the input for '..' fails — because of percent-encodings, double encodings, symbolic links, alternate path separators, Windows short names, and platform-specific syntax, **all of which produce a traversal without the literal sequence ever appearing in the input.**

So the correct procedure is: join the untrusted component to the base directory, canonicalise the resulting path fully, and then verify that the canonical result is still inside the base directory. Three steps, in that order.

Which handles symbolic links and encodings together — because you are checking the *resolved* destination rather than attempting to predict it from the input, which is the distinction that makes this approach robust where filtering is not.

And beware the race: a check-then-open sequence can have the path replaced between the check and the open — a time-of-check-to-time-of-use defect (Module 05 §3) — **so open first and verify properties of the resulting handle where the platform permits it**, using the file descriptor rather than the name.

FOR COMMANDS, PASS AN ARGUMENT ARRAY AND DO NOT USE A SHELL

A shell parses its argument string — for semicolons, pipes, backticks, command substitutions, globs, redirections, and variable expansions — **so any untrusted content anywhere in that string is code, not data.**

Passing an argument array bypasses the shell entirely: the arguments reach the target program as discrete arguments, and **no parsing of the kind above occurs at any point.** This is the `execve` interface, and most languages expose it directly.

And the remaining hazard is argument injection — an untrusted value that begins with a dash is interpreted by the called program as an option rather than as data, which has produced real vulnerabilities in programs invoked entirely without a shell. A filename of `--upload-file` is not a filename.

So: an argument array, and use the `--` separator or validate against an allowlist where the called program accepts options — and better still, call a library rather than a program, which removes the whole interface.

4 Parse, don't validate

PARSE INPUT INTO A TYPE THAT CANNOT BE INVALID, RATHER THAN CHECKING IT AND PASSING IT ON

Validation checks a value and returns the same type it was given — a string goes in and a string comes out — so **the knowledge that it was checked exists only in the programmer's head, and every subsequent function must either check it again or assume somebody did.**

Parsing converts the value into a type that can only hold valid values — an Email, a UserId, a SafePath, a ValidatedQuery — **so the type system itself carries the guarantee**, and a function requiring validated input says so in its signature.

Which eliminates Module 01 §2's step-4 problem entirely: 'validated on this path but not on that one' cannot arise, because the only way to obtain a value of the type is to go through the parser — the call graph no longer needs auditing.

And it explains why the fixes in §1 work: a prepared statement is a *parsed* query with typed holes, rather than a string that everybody hopes is well-formed — the same discipline, applied by the database driver on your behalf.

- **Validate once, at the boundary, and return a distinct type** — not a string with a comment saying it is safe, which is the pattern this discipline replaces.
- **Make the unsafe constructor private, or at least awkward**, so that a value of the parsed type cannot be fabricated by accident or by a hurried refactor — which is what makes the guarantee hold over time.
- **Push the parse as early as possible**, so that the great majority of the code never handles the untrusted representation at all — and so that the surface requiring security review shrinks to the parser.
- **And accept the narrowest input you can: a restrictive allowlist beats any blocklist (§2), and it is a parsing decision rather than a filtering one** — you are defining the valid language, not enumerating the invalid strings.
- **Which composes with Module 03 §4's type confusion fix** — the same discipline, applied against a different failure, which is why both modules converge on it. **This is the most transferable idea in the module**, and it applies in every language with a type system worth the name.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
OWASP — the injection cheat sheets (free)	§1 through §3 as current, specific guidance per destination — maintained, and the right reference to hand a team.
King — Parse, Don't Validate (free)	§4 in the original — short, and it reframes input handling permanently.
Dowd, McDonald & Schuh, chapters 8 and 17	§2 and §3's specifics — encoding confusion and path handling, in more detail than anywhere else. Library copy.

Resource	What it covers for this module
Google's safe-by-construction libraries writeup (free)	§4 at scale — types that make injection unrepresentable, deployed across a large codebase.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write a SQL injection against your own test database** and then fix it with parameterisation.
- 2 **Explain why the parameterised version cannot be injected**, mechanically.
- 3 **Take one value and escape it correctly for five HTML contexts**, and note that all five differ.
- 4 **Construct a double-decoding bypass** of a filter you wrote.
- 5 **Write a path traversal that contains no '..'** — using encoding or a symlink.
- 6 **Implement resolve-then-contain** and confirm it rejects both.
- 7 **Invoke a program with a shell and with an argument array**, and compare what a semicolon does.
- 8 **Demonstrate argument injection** with a value beginning with a dash.
- 9 **Convert one validation function into a parser** returning a distinct type.
- 10 **Make its unsafe constructor private** and fix whatever stops compiling.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State the single cause of injection and name five parsers.
- 2 Why can the downstream parser not tell data from structure?
- 3 Give the separating interface for six destinations.
- 4 Give five specific ways escaping fails.
- 5 Why is escaping positional rather than wrong?
- 6 Why does filtering for '..' fail, and what works?
- 7 What is the remaining hazard after removing the shell?
- 8 Contrast validation with parsing.
- 9 Which problem from Module 01 does parsing eliminate?
- 10 Why is a prepared statement an instance of parse-don't-validate?